Research-Based Assessment

CSE340: Computer Architecture

Instructions

In this research-based assessment, you will explore a real-world problem using the concepts learned from your textbook. Your goal is to develop a practical solution that works within the given constraints. You don't need to implement the idea, but your proposal should be grounded in theory and backed by reliable sources. Summarize your findings in a clear and concise report, no longer than **two pages (excluding References)**. **Use Times New Roman font, size 11, single column, and single spacing.**

Your Problem

Problem Number = **(YOUR ID) % 20 + 1**

Deadline

1st May 2026

Plagiarism Policy

We encourage original and creative thinking in your work. You may refer to textbooks, articles, or technical documentation; just be sure to cite all sources properly.
The accepted plagiarism (including AI plagiarism) threshold is <= 10%. Exceeding this may result in penalties.

Report Structure

Your report should follow this structured format:

- **Introduction** (3 - 4 sentences)
  - Briefly introduce the real-world system or scenario you are analyzing
  - Explain why this system is relevant to computer architecture
  - Mention the specific focus of your analysis
- **Background / Related Work** (1-2 paragraphs)
  - Summarize relevant textbook concepts
  - Mention the actual hardware architectures used in this scenario
  - Optionally, refer to industry practices or academic references
- **Methodology / System Design**
  - Propose an architectural strategy, optimization, or redesign based on your analysis
  - Include a diagram or table to explain the system architecture or data flow
  - Clearly state any assumptions
  - Justify how your solution improves performance, energy use, or system cost
  - Add some numeric comparison whenever possible
- **Discussion** (2 paragraphs)
  - Evaluate the impact of your proposal
  - Identify trade-offs or limitations
  - Mention any novel ideas or reasoning you explored during your solution design
  - *Evaluate the sustainability and impact of modern, fast-changing computer architecture and technology on society and the environment. (mandatory point - 1 single paragraph) (discussion about power consumption, long-term sustainability, effect on the environment, etc.)*
- **Conclusion** (3-4 sentences)
  - Summarize the architectural insight gained from this problem
  - Highlight how your approach benefits system design, sustainability, or user experience
  - If relevant, suggest future work
- **References**:
  - List any articles, technical specs, whitepapers, or datasheets used.
  - It is mandatory

Question 1: Investigating Memory Hierarchy in Mobile and Gaming Devices

Modern smartphones and gaming consoles achieve impressive performance despite tight power and thermal constraints. A key reason is their carefully designed memory hierarchies, which balance speed, capacity, energy efficiency, and responsiveness.

Choose a real-world platform – such as the **Apple M1 SoC, Snapdragon 8 Gen 2,** or **PlayStation 5** – and analyze how its memory system is structured to support high-performance workloads like gaming, multitasking, or video processing.

For example, the Apple M1 integrates 128KB L1 caches, a 12-24 MB L2 cache, and up to 16GB of LPDDR4X DRAM delivering ~68 GB/s bandwidth – all within a unified memory architecture shared by CPU and GPU. The PlayStation 5 pairs 16GB of GDDR6 RAM (448 GB/s with a custom SSD capable of 5.5 GB/s throughput, streaming game assets in real time without preloading large datasets into DRAM.

In your report:

- Describe the memory hierarchy of your selected device: registers, cache levels (with sizes and associativity if available), DRAM, and storage
- Explain how the architecture exploits locality principles – for example, through unified memory, fast cache access, or streaming from SSD
- Analyze tradeoffs in speed, latency, energy use, and die area among SRAM, DRAM, and NAND
- Consider whether the CPU and GPU share memory, and how cache coherence or bandwidth sharing is handled
- Support your points with real specifications and performance data from valid sources
- Finally, reflect on how memory design decisions affect the device's responsiveness, energy efficiency, longevity, and potential environmental impact.

Expected Outcome:

- Break down the memory hierarchy of a real system
- Apply concepts such as temporal/spatial locality, latency hierarchy, and cache trade-offs
- Use hardware specs to justify why certain memory types are used at different levels
- Think critically about how design choices affect both performance and long-term sustainability

Suggested Readings:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- Understanding the PS5's SSD, https://forums.anandtech.com/

Question 2: Designing a Tiered Memory System for Cloud Applications

Cloud-scale services frequently encounter performance bottlenecks due to suboptimal memory and storage hierarchies. Suppose you're part of the backend infrastructure team for a real-world application, such as a **video streaming platform** (e.g., Netflix), an **e-commerce system** (e.g., Amazon), or a **multiplayer game server** (e.g., Fortnite). Your goal is to design an improved memory and storage hierarchy that addresses one of the following real issues:

1. High latency during request spikes
2. Frequent cache misses that overload slower tiers
3. Poor utilization of SSD/NVMe or in-memory caches
4. Excessive DRAM usage by cold or stale data

In your report:

- Describe the chosen application and the problem scenario
- Propose a **tiered memory-storage architecture** with specific levels: e.g.,
  - DRAM (100 ns latency)
  - NVMe SSD (100-500 µs read latency, 2-3.5 GB/s throughput)
  - HDD or S3 storage (2-10 ms latency)
  - CDN or edge caches (targeting <50 ms response time globally)
  - In-memory cache layers (e.g., Redis, targeting <1ms latency)
- Define rules or heuristics for data placement and movement between tiers (e.g., LRU, LFU, hot/cold thresholds)
- Evaluate your design using metrics such as:
  - Expected cache hit ratio (e.g., 85-95% in Redis)
  - Average data access latency improvement
  - Potential reduction in DRAM use or cost per GB of active data
- Justify the tradeoff between speed, cost, power, and durability
- Finally, reflect on how your design could reduce energy waste or hardware overprovisioning

Expected Outcome:

- Identify a performance bottleneck in a real cloud system
- Propose and describe a multi-tier memory/storage architecture with justified numeric assumptions
- Apply textbook concepts – temporal/spatial locality, latency hierarchy, caching, etc.
- Use realistic data to justify design choices
- Evaluate their design's impact on performance, efficiency, and sustainability

Suggested Readings:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- tiered storage, ElastiCache, CloudFront, S3 Storage

Question 3: Optimizing Cache Usage in Scientific and Machine Learning Workloads

Large-scale computations in scientific applications (e.g., matrix multiplication, weather simulation, training neural networks) often face **cache bottlenecks** due to their use of large data arrays and regular memory access patterns. Your task is to investigate how modern systems (CPUs, GPUs, or edge accelerators) can improve memory locality and cache usage to speed up such workloads.

Choose a representative workload such as:

- Matrix multiplication in a BLAS library (e.g., OpenBLAS, Intel MKL)
- CNN inference or training (e.g., in TensorFlow, PyTorch)
- Data-Parallel scientific computing (e.g., FEM, CFD, or simulation kernels)

In your report:

- Explain why cache bottlenecks arise in these workloads (e.g., large working sets, strided memory access)
- Propose software-level optimization strategies, such as:
  - Loop tiling/cache blocking
  - Prefetching (manual or compiler-inserted)
  - Loop interchange or memory layout changes
- Optionally, evaluate compiler-assisted approaches (e.g., polyhedral models) or accelerator-specific tactics (e.g., tensor cores, local shared memory)
- Use a real example or pseudocode to show how your strategy improves spatial/temporal locality
- Include expected impact on cache hit ratio, latency, or performance improvement (e.g., "blocking matrix multiplication improves L1 cache hit rate from 60% to 90%)
- Finally, reflect briefly on how better cache utilization contributes to energy savings or longer hardware lifespan, especially in constrained environments like edge devices

Expected Outcome:

- Identify sources of memory inefficiency in high-computation workloads
- Apply textbook concepts like locality, blocking, associativity, and reuse distance
- Propose cache-aware code transformations
- Estimate impact on real systems (e.g., reduced DRAM access, lower latency)
- Relate memory efficiency to energy efficiency or hardware scaling

Suggested Readings:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- https://www.intel.com/content/www/us/en/developer/overview.html

Question 4: Memory-Aware Architecture for Low-Power Edge AI Devices

Many emerging applications in rural healthcare, agriculture, disaster response, or environmental monitoring require **real-time AI processing on edge devices** with severe limits on power, memory, and thermal budget. These devices typically use microcontrollers or embedded SoCs (e.g., ARM Cortex-M/R/A, RISC-V cores) with kilobytes to megabytes of memory, and no off-chip GPU.

Imagine you're tasked with designing a **memory-efficient architecture** for such an edge AI device – for instance, a wildfire detection node, a wearable health sensor, or a drone-based soil monitor – that runs a simple ML model (e.g., object detection or time-series anomaly detection)

In your report:

- Analyze the **memory hierarchy constraints** of your target platform: e.g., 64-512 KB SRAM, 1-4 MB flash, no L2 cache
- Propose an optimized **on-device memory layout** including:
  - Scratchpad RAM vs cache
  - Data buffering strategy for sensor input
  - Model weight loading/compression (e.g., quantization to int8)
- Recommend **selective caching or reuse buffering** to exploit temporal locality without wasting memory
- Suggest **emergency-mode policies** (e.g., bypassing cache, reducing precision) for power-limited critical conditions
- Estimate numeric expectations: e.g., inference must finish in <100 ms; power consumption must stay under 150 mW
- Finally, reflect on how such architectures extend device lifespan, reduce power waste, and support sustainable tech adoption in low-resource settings.

Expected Outcome:

- Design memory hierarchy layouts under embedded constraints
- Justify SRAM/flash/cache design trade-offs
- Apply quantization, compression, and software techniques (e.g., loop unrolling, static buffering)
- Address real-world constraints (latency, power, memory) using architectural principles
- Reflect on how memory-aware edge systems support energy-efficient, long-lived deployments

Suggested Readings:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- Eyeriss, Cortex, Google Coral Edge TPU

Question 5: Designing a Neuro-Adaptive Cache for Brain–Computer Interfaces

Brain-computer interfaces (BCIs) are emerging in applications ranging from VR to neurofeedback therapies. These devices must process real-time neural signals (e.g., EEG) under strict latency and power constraints, and the data often arrives in unpredictable patterns.

Design a neuro-adaptive cache system that supports real-time BCI applications on embedded processors. Your goal is to optimize both response latency and energy efficiency, despite irregular data access behaviour.

In your report:

- Choose a platform (e.g., OpenBCI, Intel Loihi, ARM Cortex-M-based headset)
- Describe the nature of the data: e.g., low-frequency idle EEG vs high-burst activity during events
- Propose a cache configuration that adapts dynamically based on brain signal behaviour:
  - Cache resizing or a selective way of prediction
  - Time-decayed eviction or burst-tracking prefetching
  - Staged buffers for idle vs burst modes
- Model how your cache would behave under different neural task loads (e.g., meditation vs active movement intent)
- Estimate constraints: e.g., cache access with 50 ns, system power budget <200 mW, burst response <10 ms
- Reflect on how adaptive memory design could improve battery life, reliability, and user safety in BCIs

Expected Outcome:

- Propose an embedded cache design tuned for irregular workloads
- Apply dynamic memory strategies (e.g., adaptive resizing, prefetching, bypass)
- Justify trade-offs in latency, power, and cache hit/miss ratio
- Demonstrate how memory design adapts to real-time neural signals
- Reflect on implications for safety-critical, wearable AI systems

Suggested Readings:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- OpenBCI, neuromorphic architectures and hardware, A Look at Loihi - Intel - Neuromorphic Chip

Question 6: Parallel Hardware Architectures in Cloud Data Centers

Cloud data centers from companies like Google, Amazon, and Microsoft must serve workloads at planetary scale – from AI training to real-time analytics. To do this, they combine thousands of parallel hardware units, including multicore CPUs, GPUs, TPUs, and memory/storage accelerators.

Your task is to analyze how cloud data centers use parallel architectures to support high-throughput, scalable computing.

In your report:

- Choose a real cloud provider (e.g., Google Cloud, AWS, Microsoft Azure)
- Describe the parallel hardware mix:
  - Multicore CPU nodes
  - GPU or TPU accelerators
  - Memory hierarchy (e.g., DRAM, HBM, NVMe SSDs per node)
- Discuss how workloads are distributed:
  - Shared memory (OpenMP) vs message-passing (MPI, gRPC)
  - Task parallelism (e.g., MapReduce, Spark), data parallelism (e.g., model training)
- Describe network interconnects (e.g., NVLink, InfiniBand, or fat-tree topology) and their role in enabling parallelism
- Analyze performance trade-offs:
  - GPU density vs cooling cost
  - Specialization (TPUs) vs generality (CPUs)
  - Throughput vs latency
- Use real metrics: e.g., Google TPU pod = 2048 chips, 11.5 exaFlops peak (as of 2023)
- Reflect on how such data center-scale parallelism supports modern computing and its energy/sustainability implications

Expected Outcome:

- Analyze architectural choices in cloud-scale systems
- Explain how different types of parallelism are used in practice
- Compare trade-offs between CPU/GPU/TPU nodes
- Relate real-world deployments to "Chapter 6" parallelism models
- Reflect on the energy and scalability implications of large parallel systems

Suggested Readings:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]
- Google TPU System Architecture, Amazon EC2, https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/Catapult_ISCA_2014.pdf

Question 7: Parallel Inference Engine for Autonomous Disaster Drones

In disaster response missions (e.g., earthquakes, floods), drones are deployed to search for survivors by processing aerial imagery using AI models. These systems must perform onboard CNN inference in real time, under power constraints, with limited connectivity, and in harsh environments.

Your task is to design a parallel processing system for autonomous drones that supports real-time object detection using SPMD-style execution and an efficient memory hierarchy.

In your report:

- Propose a hardware/software architecture for CNN inference:
  - Use lightweight parallel processors (e.g., ARM Cortex-A, RISC-V, or small GPU)
  - SPMD model: each core processes tiles of the image or CNN layers
- Suggest a memory layout that supports:
  - Streaming image tiles from the camera to SRAM or DRAM
  - Shared weights across cores
  - Efficient cache usage and tile-local reuse
- Address fault tolerance:
  - Include error detection (e.g., ECC) or core redundancy
  - Graceful fallback under partial failure
- Estimate key metrics:
  - Inference latency <150 ms per frame
  - Target energy budget: <2 W per drone
  - Cache miss rate goal: <10% during inference
- Compare rescue mode (high-speed, high-power) vs survey mode (low-speed, energy-saving)
- Reflect on how such architectures improve resilience, autonomy, and mission success

Expected Outcome:

- Design a lightweight parallel AI architecture for embedded inference
- Map CNN workloads across cores (SPMD) with localized memory
- Propose trade-offs in energy, performance, and reliability
- Address both parallelism and memory hierarchy constraints
- Link their design to real-world use cases and societal impact

Suggested Readings:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]
- SPMD, Jetson Nano, https://www.sciencedirect.com/science/article/pii/S2352864824001536

Question 8: Improving Parallel Scalability in Web Server Architectures

Small web platforms (generally used by local governments, schools, NGOs) increasingly rely on open-source web servers like Apache or Nginx to deliver services at scale. However, these systems often suffer from concurrency bottlenecks as user traffic grows.

Your task is to investigate how parallelism can improve performance in such web systems, focusing on software architecture, thread models, and memory contention.

In your report:

- Choose a real open-source server (e.g., Apache, Nginx, Node.js, or Express)
- Describe how it handles incoming requests:
  - Process/thread-per-request vs thread pools vs event loops
  - Shared memory regions (e.g., cache, session data)
- Identify a scalability bottleneck (e.g., locking, queue contention, memory pool thrashing)
- Propose a parallelism-aware fix:
  - Switch to a lock-free queue
  - Add per-thread memory queue
  - Use I/O batching, epoll/kqueue for async workloads
- Estimate improvement with metrics:
  - E.g., reduce average response latency from 250 ms to <100 ms under 1000 concurrent users
- Reflect on how small architectural changes enable scalable services on low-cost infrastructure

Expected Outcome:

- Understand real-world concurrency models in web systems
- Diagnose performance limits tied to parallel thread behaviour or shared memory
- Propose targeted, architecture-informed enhancements
- Relate solutions to parallel processing principles: scalability, contention, latency
- Consider deployment feasibility in low-resource environments

Suggested Reading:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]
- NGINX, jemalloc, Optimizing Node.js Performance

Question 9: Scaling Parallel I/O in Cloud-Based File Sharing Applications

Web applications often support large file uploads and downloads, but performance degrades sharply under concurrent access. For instance, in a file-sharing app deployed on a virtual cloud server (e.g., 4 vCPUs, 8GB RAM, 250 GB NVMe), performance collapses when 10+ users upload 100 MB files simultaneously, despite good specs and async programming.

Your task is to analyze and improve the app's parallel I/O architecture to better support concurrent heavy file operations.

In your report:

- Describe common parallel I/O architectures:
  - Thread-per-request, async event loop (Node.js), worker pools, or I/O threads
  - Blocking vs non-blocking disk access
- Analyze potential bottlenecks:
  - Thread starvation, shared queue contention, and socket backlog
  - NVMe write queue saturation, file system locks
- Propose architectural improvements:
  - Per-core I/O threads or epoll-based I/O
  - Concurrent file stream buffers, thread-safe memory allocators
  - Using chunked upload queues with back-pressure
- Estimate:
  - Concurrent user target: e.g., 20 users uploading 100 MB files in parallel
  - Target throughput: >400 MB/s sustained write
  - Response time target: <200 MB/file start delay
- Reflect on how memory, threads, and storage I/O must be coordinated, and how smart scheduling improves system resilience and cost-efficiency on cloud instances

Expected Outcome:

- Recognize the role of TLP in I/O heavy cloud services
- Identify causes of scaling failures in file handling pipelines
- Propose thread-safe and memory-coherent enhancements
- Estimate performance improvements using a parallel design
- Relate architecture to cost-aware cloud deployment and sustainability

Suggested Readings:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]
- Optimizing Node.js Performance, NVMe, Asynchronous I/O

Question 10: Diagnosing and Redesigning Memory-Parallel Bottlenecks in Smart Surveillance Systems

A retail company has deployed smart surveillance cameras across 10 store entrances. Each system runs on a Jetson Nano board with a quad-core ARM Cortex-A57 processor, 4 GB LPDDR4 RAM, and a shared 2 MB L2 cache. The board simultaneously:

- Detects people using TensorRT and OpenCV
- Records high-resolution video to a USB-connected SSD (500 MB/s)
- Uploads the video stream to the cloud (1 Gbps uplink)

During low traffic hours, the system runs smoothly. But during peak hours, when detection, recording, and uploading all happen together, the device becomes sluggish and often drops frames, even though neither the SSD nor the internet link is fully saturated.
Your task is to investigate the architecture of this system and propose improvements that resolve the performance bottleneck, while also minimizing energy usage and extending device lifespan.

In your report:

- Analyze how concurrent video tasks stress the system: TensorRT inference using shared cache, DRAM pressure from large frame buffers, USB SSD writes competing with inference reads, and Thread-level contention between I/O and compute
- Identify architectural bottlenecks: Cache misses or poor locality in OpenCV pipelines, Thread starvation or lack of real-time prioritization, Excessive memory writes that wear down the SSD
- Propose an architectural or a software solution, such as: Tiling inference into memory-fit patches, Cache-aware task scheduling (e.g., pin uploads to one core), Write-combined buffers or staggered I/O scheduling, Real-time thread prioritization or task offloading
- Estimate improvements: e.g., reduce frame drops by 70% during peak load, 2x longer SSD lifespan from reduced redundant writes, 30% lower energy draw via better memory access patterns
- Reflect on how architecture-aware design can improve long-term sustainability, reduce e-waste, and enhance safety in real-world public-facing systems

Expected Outcome:

- Analyze embedded parallel workloads under mixed compute and I/O
- Propose memory hierarchy-aware scheduling or caching solutions
- Understand how concurrency and DRAM/cache limits interact in edge AI
- Relate system efficiency to environmental and deployment sustainability

Suggested Reading:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]

Question 11: Optimizing Embedded Memory and Control for Field Diagnostics in Bio-Labs

A health tech company is developing a portable molecular diagnostic device for use in rural clinics and mobile labs. The device performs biochemical reactions (e.g., for virus detection or genetic markers) in 4-6 microfluidic chambers. Each chamber operates independently, but must be controlled, monitored, and logged by an onboard embedded system.
Hardware specs:

- Single-core ARM Cortex-M4, 200 MHz
- 32 KB SRAM, 512 KB flash, no cache
- Reaction controller, thermal regulators, sensor inputs
- Flash logger via USB; max power: <300mW

Each reaction generates ~6KB of data every 5-15 minutes. During multi-sample runs, engineers observe:

- SRAM overflows due to logging queues
- Control logic stalls during flash writes
- Missed temperature control windows, affecting diagnostic accuracy

In your report:

- Identify where memory limitations and timing conflicts arise between data logging and chamber control
- Propose architecture-level improvements: fixed memory partitioning, buffered batching, priority-based scheduling, or other control strategies
- Estimate: how many reactions can run reliably if 20KB SRAM is used for logging, and flash writes take 1.5s per log batch?
- Reflect on how such designs enable sustainable, low-cost diagnostics in remote areas, minimizing hardware waste and power consumption.

Expected Outcome:

- Diagnose concurrency and memory issues in an embedded real-time system
- Propose optimizations grounded in architecture
- Justify the design using numeric assumptions
- Reflect on computing as an enabler of health equity and sustainable medical technology

Suggested Reading:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]

Question 12: Designing a Memory-Efficient Smart Classroom Assistant for Rural Schools

A low-cost "Smart Board" system is being deployed in rural classrooms with unreliable internet. Each unit includes:

- A dual-core RISC-V processor @ 1 GHz
- 1 MB L2 cache, 512 MB LPDDR3 RAM
- A local camera and microphone for recording the teacher's session
- An onboard neural net for transcribing blackboard writing and speech
- Local storage (32 GB flash) for auto-saving class logs and generating offline feedback reports

To ensure privacy and offline access, everything must run locally. During peak load (e.g., board writing + voice input + logging), the system experiences:

- Transcription lags
- Buffer overflows during long sessions
- SSD wear due to frequent small writes

In your report:

- Explain where memory contention or concurrency issues arise across audio/video pipelines, inference, and logging. Consider how cache reuse and flash access patterns affect performance.
- Propose smart buffering, tasking partitioning (e.g., voice vs vision thread pinning), memory pooling for temporary logs, or SRAM-based intermediate storage to defer flash writes.
- Given that transcription uses 200 MB of RAM continuously, each log write = 4KB (occurs every 3 sec), flash write latency = 0.4 ms. Estimate how long the system can operate without hitting a memory ceiling or overwhelming the SSD, and how batching logs (every 30 seconds) affects performance.
- Discuss how efficient architecture enables long-term, low-energy, low-maintenance educational tools, promoting equitable tech access without constant upgrades or cloud dependence.

Expected Outcome:

- Design memory and task management for constrained, real-time systems
- Estimate bottlenecks and endurance under real usage
- Reflect on how architecture enables inclusion, privacy, and sustainable deployment in education

Suggested Reading:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]

Question 13: Choosing Between Competing Architectures for Community Wi-Fi Edge Nodes

A non-profit organization is rolling out community Wi-Fi hotspots in underserved regions. These edge nodes not only route internet traffic, but also cache local educational videos, perform basic intrusion detection, and log network activity for accountability. They must operate off-grid for 10 hours/day and are deployed in areas with intermittent solar power and no tech support.
Two prototypes are available:

- System A
  - Dual-core RISC-V, 1.2 GHz
  - 1 MB L2 cache, 512 MB DDR3
  - Local storage: 128MB NOR flash
  - Power: 2.5 W average
- System B
  - Quad-core ARM Cortex-A53, 1.5 GHz
  - 512 KB L2 cache, 1 GB LPDDR4
  - 64 GB eMMC
  - Power: 5.5 W average

Your task is to analyze and critically evaluate both designs for long-term deployment.

In your report:

- Which system is better suited for memory-constrained, multi-threaded workloads like caching + IDS? Discuss trade-offs in memory hierarchy, concurrency potential, and storage reliability.
- What might go wrong in either system under daily usage? Where could the cache be insufficient or RAM overused? Could flash wear or power budget become critical?
- Using what you learned, propose a revised system that combines the strengths of both, or improves on them, e.g., task offloading, selective caching, staggered scheduling.
- Which design is more sustainable over 5 years in harsh environments (heat, dust, energy limits)? How does efficient architecture promote digital inclusion with minimal e-waste?

Expected Outcome:

- Evaluate real architectural trade-offs between memory, concurrency, and sustainability
- Propose design improvements based on workload and power constraints
- Quantitatively justify their preference
- Reflect on computing for social infrastructure and environmental resilience

Suggested Reading:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]

Question 14: Evaluating Diagnostic Edge Devices for Remote Clinics

A healthcare NGO is testing two portable diagnostic kits for use in off-grid clinics. These devices process basic blood chemistry panels and display results locally. Both platforms are designed to collect data from sensors, run ML models for abnormality detection, and store and sync results with hospital servers when a connection is available. Here are the two specs:

- Device Alpha
  - Quad-core ARM Cortex-A55, 1.8 GHz
  - 1MB shared L2 cache, 2 GB LPDDR4 RAM
  - 128 GB SSD storage
  - 5W average power draw
- Device Beta
  - Dual-core RISC-V, 1.2 GHz
  - 2 MB shared L2 cache, 512 MB LPDDR3 RAM
  - 64 MB NOR flash (no local storage beyond logs)
  - 1.8 W average power draw

Your task is to evaluate these two specs for deployment in rural diagnostic centers with limited electricity, no tech support, and data upload windows of just 30 minutes per day.

In your report:

- Which design better supports the concurrent tasks of sensor data collection, inference, and secure logging? Discuss memory bandwidth, cache usage, and task scheduling capacity
- What risks emerge in either system under prolonged offline operation? What are the trade-offs in model performance, log retention, and power resilience?
- Suggest a revised version that combines the strengths of both or improves on them. Could batching inference or layering cache-efficient logging reduce memory stress?
- Which device is more viable over 3-5 years in the field? Consider power constraints, component wear, and local repairability. Reflect on how architectural design influences healthcare equity and access to technology.

Expected Outcome:

- Analyze design trade-offs in embedded edge diagnostics
- Propose efficient memory and scheduling techniques under power limits
- Support decisions with technical and societal reasoning
- Connect architecture to sustainable, inclusive healthcare delivery

Suggested Reading:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]

Question 15: Scaling the Architecture of a Village Weather Station Over Time

A small rural school initially deployed a simple weather station with a temperature sensor, humidity sensor, and a microcontroller logging data to an SD card once every hour. The original system used an 8-bit microcontroller (16 MHz), 2KB SRAM, 32 KB flash, no cache, no OS, single-threaded loop, 100 mAh battery + solar cell for Power, cost: <1200 tk.
After two years, the community asks for: 10x faster sampling (every 6 minutes), Real-time alerting for storms (via buzzer and LED logic), A Bluetooth-based mobile app to sync logs monthly, Offline chart rendering on a tiny 1.8" screen, and Longer operation time on the same power budget.

Your task is to trace the architectural evolution required to support these upgrades while respecting constraints of ultra-low budget, tight energy envelope, limited compute, and memory.

In your report:

- Highlight the memory and concurrency limitations of the original system. What fails if we just add features? Where do memory, timing, and energy break down?
- Propose a revised architecture. Pick a processor class, cache structure (if any), and memory hierarchy. Propose changes like task scheduling, event-driven design, ring buffers, or timer interrupts to support the expanded workload.
- Estimate how each new feature (e.g., chart rendering, Bluetooth sync) adds to RAM/Flash/energy demand. Suggest batching or deferred strategies to keep within 100 mAh/day.
- Reflect on how gradual system growth can be anticipated with scalable architecture choices. How does thoughtful design avoid e-waste, support community resilience, and reduce the need for expensive upgrades?

Expected Outcome:

- Understand system design across time and feature growth
- Propose scalable architectures under budget/energy constraints
- Apply concurrency and memory strategies incrementally
- Reflect on long-term sustainability through architectural foresight

Suggested Reading:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]

Question 16: Can a Self-Driving Car's Architecture Power a Home Robot?

A robotics company wants to repurpose a previously developed self-driving vehicle compute module for a new household cleaning robot. The vehicle module includes: Octa-core CPU, 4 MB L2 cache, 16 GB LPDDR5 RAM, 1.2 TOPS AI accelerator, Dual 4K camera input + LIDAR, NVMe SSD (1 TB), 100 W peak power draw, and it is used for 3D mapping, object recognition, path planning, and fusion.
For the home robot, goals are: Indoor SLAM (2D), person tracking and obstacle avoidance, dust mapping and route optimization, 10-hour battery life, <10W average consumption, local log storage and mobile app sync once a day.

In your report:

- Evaluate the original architecture. What architectural features (e.g., memory hierarchy, parallel task split) are overkill or unsuitable for the home context? Which parts are reusable?
- Propose a downsized architecture or reconfiguration for the home robot, with justification. Should AI acceleration be disabled? Can DRAM or SSD use be scaled down?
- Compare expected energy, memory, and latency performance before and after simplification. E.g., how much RAM is truly needed for 2D mapping and person tracking? How would reducing cache or core count affect task concurrency?
- What insights does this case offer about adapting high-performance systems for resource-limited applications? When is a redesign/simplification approach more appropriate than a direct downscaling of the original architecture?

Expected Outcome:

- Practice abstraction and repurposing of architectures
- Understand memory-performance-power tradeoffs across domains
- Justify simplification through numeric and qualitative reasoning
- Reflect on the ethics and value of reuse vs reinvention in computing

Suggested Reading:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]

Question 17: Memory and Parallelism Challenges in AI-Generated Fashion Mirrors

A design studio sets up small pop-up booths in malls where customers can "try on" algorithmically generated clothing styles using an AI fashion mirror. Each booth has:
A quad-core ARM Cortex-A76 @ 2 GHz, 1.5 MB L2 cache, 6 GB LPDDR4 RAM, RGB camera, Depth sensor, On-device generative model, Local SSD(64 GB), passive cooling (no fans). The system performs live body segmentation + posture detection, real-time generation of clothing overlays (2-3/sec), and optional local save of best outfit renderings (upon user tap).

Issues observed during pilot runs:

- Lag when users rotate quickly
- Generated clothing flickers when the system gets overloaded
- Overheating causes performance throttling
- SSD wear shows abnormal growth in kiosks with user turnover

In your report:

- Diagnose how memory contention and task concurrency lead to delays and SSD stress
- Propose optimizations like cache partitioning, frame buffering, and deferred writes
- Estimate: Given 3 MB/frame, 10MB/save, and 100 TBW SSD lifespan, how long before failure under different write strategies?
- Reflect on how architectural tuning supports sustainable, local AI in creative tech contexts.

Expected Outcome:

- Evaluate real-time edge AI under user-driven constraints
- Propose a memory hierarchy and concurrency improvements
- Justify sustainability through technical insight

Suggested Reading:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]

Question 18: Cache-Conscious Design for Real-Time Molecular Simulation on Laptops

A bioinformatics research team is building a real-time protein folding visualizer for classrooms. The software runs on student laptops (spec: Intel Core i5, 4 cores, 3 MB L3 cache, 8 GB RAM) and performs:

- Real time molecular dynamics (MD) simulation (position updates every 2 ms)
- Visualization of 3D protein structures
- Optional live export of structural states for later replay

However, users report:

- Simulation slows significantly after ~1000 atoms
- Frame jitter and UI lag appear during recording
- CPU utilization fluctuates despite parallel code (OpenMP)

In your report:

- Analyze how matrix-heavy workloads (e.g., force calculation and position updates) create cache contention with UI and export tasks
- Propose architecture-level changes such as loop tiling, cache blocking, prefetching, or simulation-UI decoupling strategies
- Estimate: Given that each atom needs 120 bytes/frame and matrix ops touch 20% of total atoms per update, model how data fits (or fails) within a 3 MB cache and what cache reuse techniques help.
- Reflect on how optimizing cache usage in educational or research tools reduces energy waste, extends laptop usability, or democratizes access to advanced simulations.

Expected Outcome:

- Apply memory hierarchy theory to real scientific code
- Learn software-level cache tuning (e.g., loop transformations)
- Quantify performance limits tied to cache size and layout
- Evaluate the sustainability of performance in resource-constrained environments

Suggested Reading:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]

Question 19: Evaluating Architecture Trade-offs Between Apple M1 and Intel i7 for Parallel Data Pipelines

A startup is building a lightweight media analytics tool that processes large batches of user-uploaded videos (e.g., for compression, tagging, and keyword detection). Two developers test the same multithreaded C++ code on:

- MacBook Air (M1 chip)
  - 4 performance + 4 efficiency cores (big.LITTLE)
  - 16 MB shared L2 cache, unified memory (8 GB LPDDR4X)
  - High memory bandwidth (68 GB/s). Low power consumption
- Windows laptop (Intel Core i7-11800H)
  - 8 identical cores, 24 MB L3 cache, 8 MB L2 total
  - 8 GB DDR4 RAM (non-unified), discrete GPU
  - Moderate memory bandwidth (~25 GB/s), higher power draw

They run the same workload: Decode and tag 10-minute videos in parallel (4 threads), store temporary feature maps in memory, export tags + compressed previews to disk.
Surprisingly, the M1 system is faster and cooler, but occasionally stutters. The Intel system is consistent, but draws more power and shows memory usage spikes.

In your report:

- Analyze architectural differences: unified vs discrete memory, cache layout, core design, bandwidth
- Explain why M1 performs well in some areas but struggles with stability
- Suggest optimizations tailored to each system: e.g., batching, thread pinning, memory reuse
- Reflect on how these trade-offs matter for sustainable, portable software in data-heavy applications

Expected Outcome:

- Compare two mainstream CPU architectures in depth
- Link memory hierarchy and cache layout to software behaviour
- Propose design or tuning strategies based on hardware limits
- Reflect on energy-performance tradeoffs in cross-platform development

Suggested Reading:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]

Question 20: Preserving Interactive Digital Art with Architecture-Conscious Edge Devices

A modern art museum showcases large-scale, interactive installations that react to motion, sound, and touch. To preserve them for future generations, curators install edge-based capture and behaviour logging units that must:

- Record audience interactions (motion sensors + microphones)
- Log system states (e.g., LED patterns, audio output triggers)
- Store media + behavior models locally (no cloud allowed)
- Operate silently with no visual disruption (no fans or lights)
- Run continuously for 8-10 hours daily

Hardware includes: Quad-core low-power ARM SoC @ 1.4 GHz, 512 KB L2 cache, 2 GB LPDDR3 RAM, Passive-cooled aluminum case, 128 GB SSD, no external connectivity during exhibit time.
After 2 months:

- Some systems begin skipping input events
- SSD access becomes erratic due to metadata logging
- Memory usage increases steadily, requiring manual reboots

In your report:

- Analyze why architectural oversights (cache overflow, memory leakage, I/O jitter) hurt real-time behaviour logging
- Propose redesigns like: prioritized memory zones, log coalescing, real-time-safe memory management, or behavior chunking
- Estimate: if each event = 2 KB and logs occur every 0.5s per user, with 100 users/day, how long until memory saturation? How can batching and caching extend uptime?
- Reflect on how careful architectural planning protects fragile digital heritage and reduces maintenance needs in cultural spaces.

Expected Outcome:

- Apply architectural concepts in a preservation-focused context
- Design systems mindful of physical and temporal constraints
- Use quantitative modeling to justify reliability improvements
- Reflect on sustainability and cultural responsibility in computing

Suggested Reading:

- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 5: Large and Fast - Exploiting Memory Hierarchy]
- Patterson & Hennessy – Computer Organization and Design: RISC-V Edition [Chapter 6: Parallel Processors from Client to Cloud]